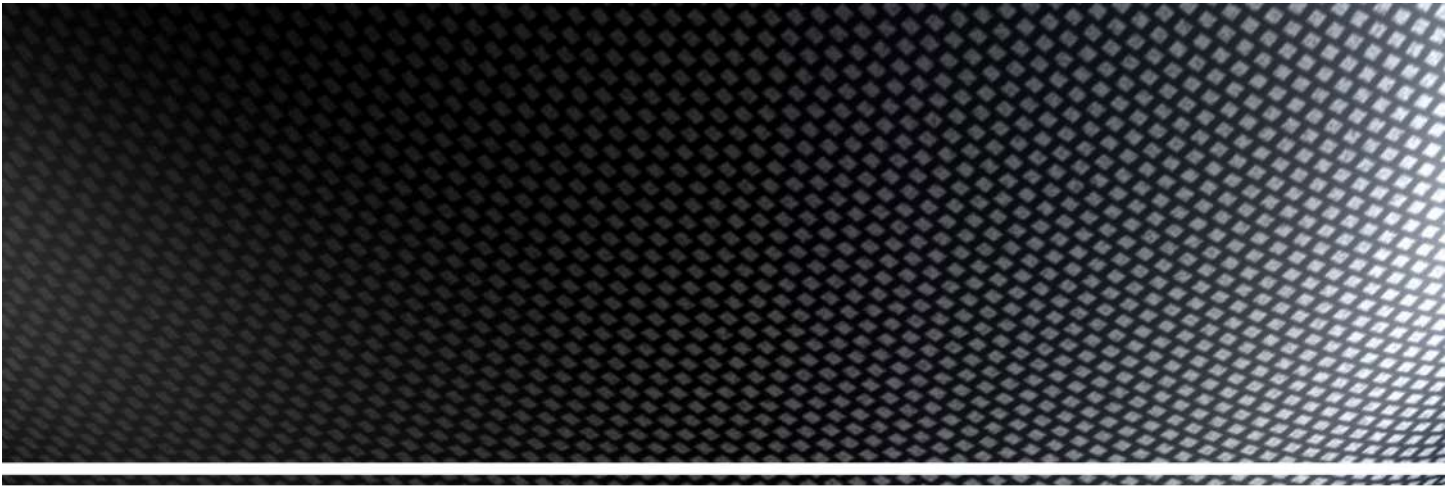




Dialogs

AlertDialog

AlertDialogs

Un **Dialog** es una pequeña ventana que informa al usuario y requiere de él una acción de confirmación. No ocupa toda la pantalla y suelen ser modales, requiere la acción para cerrarse.

La clase base es **Dialog**, pero **se deben instanciar siempre clases hijas** como, **AlertDialog**, **DatePickerDialog** ...



Se debe usar un **DialogFragment** como contenedor del **Dialog** y llamar a sus métodos en vez de a los de **Dialog**. Esto asegura la correcta evolución, el ciclo de vida, cambio de orientación y permite hacerlo embebido en otros UI.

Creamos un nuevo objeto de nuestra clase **MyDialogFragment** y llamamos al método **.show()** para mostrar el Dialogo.

Con la clase **AlertDialog** podemos montar una gran variedad de diálogos.

Se compone de:

Título:

Es opcional y para la mayoría de **AlertDialogs** informativos o intuitivos no es necesario.

Contenido:

Puede mostrar un texto, una lista o un custom layout

Action Buttons:

No debería haber más de tres botones en un diálogo.

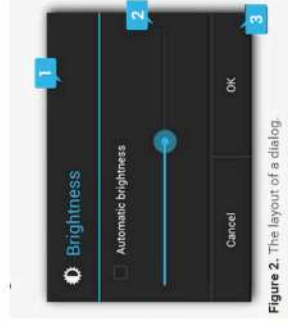


Figure 2. The layout of a dialog.

AlertDialogs

Para crear el **AlertDialog** usamos la clase **AlertDialog.Builder** que proporciona el API para crear estos componentes.

Para construir un **AlertDialog**:

`getActivity()` es una función de la clase **Fragment**. Es necesario llamarla para obtener la referencia a la **Activity** en la que se inserta o lanza el **Dialog**.

```
// Instantiate an AlertDialog.Builder with its constructor
AlertDialog.Builder builder = new AlertDialog.Builder(getActivity());

// Chain together various setter methods to set the dialog characteristics
builder.setMessage(R.string.dialog_message)
    .setTitle(R.string.dialog_title);

// Add the buttons
builder.setPositiveButton(R.string.ok, new DialogInterface.OnClickListener() {
    public void onClick(DialogInterface dialog, int id) {4
        // User clicked OK button
    }
});

builder.setNegativeButton(R.string.cancel, new DialogInterface.OnClickListener() {
    public void onClick(DialogInterface dialog, int id) {
        // User cancelled the dialog
    }
});

// Set other dialog properties
...

// Create the AlertDialog
AlertDialog dialog = builder.create();
```


AlertDialogs – Respondiendo a los eventos onClick()

Dos opciones en función de desde dónde queremos responder:

- Respondemos en nuestro **Dialog** object, donde podemos realizar las acciones necesarias para la respuesta a los eventos de los botones de usuario **onClick()**, es decir, **en el builder definimos el código de respuesta** a los eventos.
- Respondemos desde la Activity o el Fragment desde donde lanzamos el Dialogo. Deberemos recibir, en el Activity o el Fragment, los eventos **onClick()** y entonces gestionar la respuesta del usuario. Es decir, que **propaguemos el evento** desde el Dialogo a la Activity (o Fragment).

Para ello tenemos que:

1. Definir un método de interface para cada evento click.
2. Forzar que el host component implemente la interfaz
3. Implementar la interfaz en el host component
4. Enviar los eventos a través de la interfaz

AlertDialogs

1. Definir un método de interface para cada evento click.

En nuestro **MyDialogFragment**:

```
/* The activity that creates an instance of this dialog fragment must
 * implement this interface in order to receive event callbacks.
 * Each method passes the DialogFragment in case the host needs to query it. */
public interface MyDialogListener {
    public void onDialogPositiveClick(DialogFragment dialog);
    public void onDialogNegativeClick(DialogFragment dialog);
}
// Use this instance of the interface to deliver action events
MyDialogListener mListener;
```

2. Forzar que el host component implemente la interfaz

```
@Override
public void onAttach(Activity activity) {
    super.onAttach(activity);
    // Verify that the host activity implements the callback interface
    try {
        // Instantiate the MyDialogListener so we can send events to the host
        mListener = (MyDialogListener) activity;
    }
    catch (ClassCastException e) {
        // The activity doesn't implement the interface, throw exception
        throw new ClassCastException(activity.toString() + " must implement MyDialogListener");
    }
}
```

onAttach() es un método de **Fragment** que se llama cuando un **fragment** se vincula por primera vez a su contexto. Tras esto se llamará a su **onCreate()** (del

AlertDialogs

3. Implementar la interfaz en el host component

La actividad host, crea una instancia del diálogo con el constructor de `MyDialogFragment` y recibe los eventos mediante la implementación de la interfaz `MyDialogListener`.

```
public class MainActivity extends FragmentActivity
    implements MyDialogFragment.MyDialogListener{

    ...

    public void showMyDialog() { //A function to show the Dialog
        // Create an instance of the dialog fragment and show it
        DialogFragment dialog = new MyDialogFragment();
        dialog.show(getSupportFragmentManager(), "MyDialogFragment");
    }

    // The dialog fragment receives a reference to this Activity through the
    // Fragment.onAttach() callback, which it uses to call the following methods
    // defined by the MyDialogFragment.MyDialogListener interface
    public void onDialogPositiveClick(DialogFragment dialog) {
        // User touched the dialog's positive button
        ...
    }

    public void onDialogNegativeClick(DialogFragment dialog) {
        // User touched the dialog's negative button
        ...
    }
}
```


AlertDialogs

4. Enviar los eventos a través de la interfaz
Puesto que la Hosting Activity implementa el interfaz MyDialogListener, el Dialog puede enviarle los eventos a través de la interfaz.

```
builder.setMessage(R.string.dialog_fire_missiles)
    .setPositiveButton(R.string.fire, new DialogInterface.OnClickListener() {
        public void onClick(DialogInterface dialog, int id) {
            // Send the positive button event back to the host activity
            mListener.onDialogPositiveClick(MyDialogFragment.this);
        }
    })
    .setNegativeButton(R.string.cancel, new DialogInterface.OnClickListener() {
        public void onClick(DialogInterface dialog, int id) {
            // Send the negative button event back to the host activity
            mListener.onDialogNegativeClick(MyDialogFragment.this);
        }
    });
```

AlertDialogs

Otro ejemplo con un TimePicker

Ahora usamos un fragment

Sin embargo ahora el evento lo propagamos hasta el Fragment

```
public static class TimePickerFragment extends DialogFragment
    implements TimePickerDialog.OnTimeSetListener {
```

```

@Override
public Dialog onCreateDialog(Bundle savedInstanceState) {
    // Use the current time as the default values for the picker
    final Calendar c = Calendar.getInstance();
    int hour = c.get(Calendar.HOUR_OF_DAY);
    int minute = c.get(Calendar.MINUTE);

    // Create a new instance of TimePickerDialog and return it
    return new TimePickerDialog(getActivity(), this, hour, minute,
                               DateFormat.is24HourFormat(getActivity()));
}

public void onTimeSet(TimePicker view, int hourOfDay, int minute) {
    // Do something with the time chosen by the user
}
}
```

El callback que recibe en el Fragment el evento onClick que se produce en el botón del Picker